

Conclusion & Future Work

1. Overall Achievement

The project successfully evolved into a functional full-stack educational platform that combines programming practice, automated code execution, AI-assisted learning, teacher-controlled assessment, structured grading, analytics, and academic integrity mechanisms within a single system. Instead of remaining a simple assignment submission tool, the platform provides separate workflows for students, teachers, and coordinator-level academic management.

One of the main achievements of the project is the integration of AI as a controlled educational assistant rather than an unrestricted problem solver. On the student side, the AI Mentor supports learning by giving hints, explaining errors, and guiding students through programming concepts. On the teacher side, AI is used to reduce workload through problem variation generation, rubric generation, and grading suggestions. This dual use of AI supports both learning and assessment while keeping the teacher in control of final academic decisions.

The system also achieved a strong connection between its major technical components. The frontend provides role-based interfaces and an IDE-like coding workspace, while the backend handles authentication, authorization, assignment logic, code execution requests, AI service communication, grading, analytics, and coordinator-level operations. The database supports these workflows by storing users, problems, test cases, assignments, submissions, rubrics, grades, AI interactions, flashcards, and exam-related records. Together, these components form a coherent platform rather than a set of disconnected features.

Another important achievement is the inclusion of academic integrity mechanisms. Features such as hidden test cases, hidden reference solutions, teacher-controlled Exam Mode, submission history, role-based authorization, and teacher registration approval help the platform support both practice-oriented learning and controlled assessment scenarios. This balance is especially important because the project aims to use AI in education without allowing it to replace student problem solving.

Overall, the project achieved more than the minimum expected scope. It demonstrates a working prototype of an AI-assisted programming education and assessment platform, with meaningful support for students, teachers, and academic coordinators. The current

implementation provides a solid foundation for future improvements in production security, scalability, AI safety validation, and real classroom evaluation.

2. Key Achievements

The project achieved several important outcomes across the frontend, backend, AI, database, and deployment layers. These achievements show that the platform became more than a basic programming assignment system and developed into an integrated educational environment.

One of the main achievements is the creation of a role-based learning and assessment workflow. Students, teachers, and the Coordinator Teacher have separate interfaces and permissions according to their responsibilities. Students can solve programming problems, run and submit code, receive feedback, use AI Mentor support, review tutorials, track progress, and study with flashcards. Teachers can manage assignments, enroll students, create and organize questions, review submissions, generate rubrics, use AI-assisted grading suggestions, and analyze class performance. The Coordinator Teacher role extends the system by supporting teacher approval and student-to-teacher assignment workflows.

Another achievement is the successful integration of browser-based programming practice with automated code execution. The platform allows students to write code directly in the browser, execute it, see compiler or runtime feedback, and submit solutions for evaluation. This makes the system closer to an educational online judge while still keeping the user experience focused on learning rather than only pass/fail results.

The AI integration is also a major achievement of the project. Instead of using AI as a direct answer generator, the platform uses AI as a controlled mentor for students and as a productivity assistant for teachers. The AI Mentor can guide students through hints, explanations, debugging support, and concept clarification. On the teacher side, AI supports question variation generation, rubric generation, and grading suggestions. This design supports the original goal of using AI to improve education while still keeping students and teachers responsible for the learning and assessment process.

The project also achieved a strong level of system integration. The frontend, backend, database, code execution layer, and AI services work together through structured workflows. The frontend provides role-specific user experiences, the backend manages authentication, authorization, business logic, AI communication, execution requests, grading, and analytics, while the database stores the core educational data such as

users, problems, assignments, test cases, submissions, rubrics, grades, AI interactions, flashcards, and exam-related records.

Academic integrity is another important achievement. The platform includes mechanisms such as hidden test cases, hidden reference solutions, role-based authorization, submission history, teacher registration approval, and Exam Mode. Exam Mode is especially important because it allows the same platform to support two different educational contexts: AI-supported learning during practice and homework, and restricted AI access during exams.

Finally, the project reached a deployable prototype level through Docker-based and cloud-oriented infrastructure. This is important because the system was not only implemented as isolated local components, but also organized in a way that can be demonstrated and extended as a full-stack application. Although further production hardening is still needed, the current implementation provides a strong foundation for future institutional use.

3. Limitations

Although the project reached a functional and integrated prototype level, several limitations remain before the platform can be considered fully ready for large-scale institutional or production use. These limitations do not reduce the value of the current implementation, but they show the areas that require further validation, hardening, and refinement.

One important limitation is related to production readiness. The current system demonstrates that the frontend, backend, database, AI services, and code execution layer can work together as a complete platform. However, production deployment would require stronger environment configuration, database backup and recovery strategies, monitoring, logging, HTTPS configuration, and stricter separation between development and production settings. Without these improvements, the system is more suitable as a strong prototype or demonstration environment than as a fully production-ready institutional service.

Security and authorization also remain important areas for improvement. The implementation includes role-based access control, hidden reference solutions, hidden test cases, Exam Mode, teacher approval, and student-teacher assignment control as academic integrity mechanisms. However, these mechanisms must be consistently enforced on the backend side, not only through frontend visibility or interface restrictions. Sensitive data such as reference solutions, hidden tests, grades, AI logs,

and student submissions must be protected with strict ownership checks and carefully reviewed API responses.

Another limitation is the reliability and safety of AI assistance. The original project proposal already identified the risk that AI may accidentally provide complete answers instead of guidance. The current system addresses this risk through the idea of AI as a mentor rather than a solver, but this still requires continuous testing and validation. AI responses should be checked to ensure that they provide hints, explanations, and learning support without directly solving assignments. This is especially important in programming education, where even a small code suggestion can sometimes reveal the full solution.

The Exam Mode mechanism is a strong academic integrity feature, but it should also be seen as a limitation if used alone. The implementation correctly disables AI Mentor access during exam scenarios, allowing the platform to separate learning mode from assessment mode. However, real exam integrity requires more than disabling AI. Backend enforcement, activity logging, violation tracking, secure submission rules, and careful review of student behavior would be necessary for higher-stakes assessment environments.

Scalability is another limitation. The platform includes resource-heavy components such as browser-based code editing, automated code execution, AI requests, analytics, and teacher dashboards. The proposal also identified scalability for multiple simultaneous classroom or exam users as a challenging issue. Therefore, before real classroom deployment, the system should be tested under concurrent users, simultaneous code executions, and multiple AI requests. Rate limiting, request queues, execution sandbox limits, and performance monitoring would be important future improvements.

The project also has limitations in terms of evaluation. The implementation report lists functional validation scenarios for student, teacher, and coordinator workflows, such as login, assignment creation, code execution, AI Mentor restriction, grading, analytics, and teacher approval. However, functional testing alone does not fully measure educational effectiveness. The proposal mentions usability tests, student learning outcomes, time-to-solve comparisons, teacher workload reduction, and user satisfaction surveys as possible evaluation methods. These types of user-centered and educational evaluations would be needed to prove the platform's real impact in a classroom setting.

Finally, some limitations are related to maintainability and long-term extensibility. As the project expanded, the frontend, backend, AI services, database models, grading workflows, analytics, and exam features became more complex. This complexity is expected for a full-stack educational platform, but future development would benefit from more automated tests, clearer API contracts, shared TypeScript/database types,

stronger documentation, and modular refactoring of large frontend and backend components.

Overall, the current limitations mainly reflect the difference between a successful academic prototype and a production-ready educational platform. The system already demonstrates the core concept effectively, but future work should focus on security hardening, AI safety validation, scalability testing, deployment reliability, maintainability, and real classroom evaluation.

4. Lessons Learned

One of the most important lessons learned from this project is that integrating AI into an educational platform is not only a model integration problem. The AI component becomes useful only when it is supported by proper context management, role-based access control, prompt design, validation, logging, and user interface design. In this project, AI was designed as a mentor for students and as a productivity assistant for teachers, rather than as an autonomous solver or grader. This showed that the educational value of AI depends strongly on how the surrounding system controls and guides its behavior.

Another key lesson is that academic integrity must be considered as an architectural requirement from the beginning. Since the platform allows students to receive AI support during programming practice, the system also needs mechanisms to prevent misuse during assessment. Features such as Exam Mode, hidden test cases, hidden reference solutions, teacher-controlled assignments, submission history, and role-based access control were therefore essential parts of the system design. The implementation report also emphasizes Exam Mode as a way to separate learning scenarios from exam scenarios, where AI Mentor access is disabled to preserve academic integrity.

The project also showed the importance of designing AI as a human-in-the-loop tool. On the teacher side, AI can generate problem variations, rubrics, and grading suggestions, but the final academic decision must remain under teacher control. This is important because AI-generated outputs can be useful, but they still require human review for correctness, fairness, and suitability. The proposal also identified risks such as AI accidentally providing complete answers and the need to minimize AI bias in evaluation, which confirms that AI support should be controlled and reviewed rather than blindly trusted.

Another lesson learned is that full-stack integration is more difficult than implementing isolated features. A feature can look complete on the frontend, but it also needs correct

backend authorization, database support, API consistency, and security checks. For example, assignment management, code execution, AI Mentor support, grading, analytics, and exam rules all depend on multiple layers working together. This made it clear that a successful educational platform requires coherent communication between frontend components, backend services, database models, execution systems, and AI services.

The project also highlighted the importance of balancing usability with control. Students need a simple and supportive coding environment, while teachers need powerful management tools for assignments, grading, rubrics, question banks, and analytics. At the same time, the system must prevent students from accessing teacher-only data, hidden tests, reference solutions, or AI support during restricted exam periods. This balance between ease of use and strict control became one of the central design challenges of the platform.

Another important lesson is that deployment and infrastructure decisions affect the quality of the whole system. Docker-based organization and cloud-oriented deployment made it possible to run the platform as a connected full-stack application rather than separate local modules. However, this also showed that production readiness requires additional concerns such as environment configuration, database backup, HTTPS, monitoring, logging, and resource limits for code execution and AI requests.

The project also demonstrated that educational effectiveness cannot be proven only by implementing features. Although the platform provides AI Mentor support, analytics, feedback flashcards, grading tools, and teacher dashboards, their real value should eventually be measured through user testing. The original proposal mentioned evaluation methods such as usability tests, learning outcome comparison, teacher workload reduction, and user satisfaction surveys. This showed that future validation should include not only technical testing, but also educational and user-centered evaluation.

Overall, the main lesson learned is that building an AI-assisted programming education platform requires both technical and pedagogical thinking. The project combined software engineering, AI integration, database design, security, cloud deployment, and educational assessment workflows. This made the system more complex than a standard programming assignment platform, but it also made the final prototype more meaningful as a foundation for future classroom-oriented development.

5. Future Work

Although the current system provides a strong foundation for an AI-assisted programming education and assessment platform, several improvements can be made in future work to increase its reliability, scalability, security, and educational effectiveness.

One important future direction is production hardening. The current prototype demonstrates that the frontend, backend, database, AI services, and code execution layer can work together as an integrated system. However, before the platform is used in a real institutional environment, deployment-related improvements should be completed. These include configuring a proper production environment, enabling HTTPS, improving reverse proxy settings, separating development and production configurations, adding monitoring and logging tools, and creating a reliable database backup and recovery strategy.

Another major future work area is security and authorization improvement. Since the platform stores sensitive educational data such as submissions, grades, hidden test cases, reference solutions, AI interaction logs, and student analytics, all access control rules should be reviewed carefully. Backend-side ownership checks should be strengthened so that users can only access the data they are authorized to view. Teacher-only data, hidden reference solutions, and hidden test cases should never be exposed to students through API responses. This is especially important because the implementation already relies on academic integrity mechanisms such as role-based authorization, hidden test cases, hidden reference solutions, and Exam Mode.

AI safety validation is another important future direction. The project was designed around the idea that AI should guide students without directly solving problems for them. The original proposal also identified the risk that AI may accidentally provide complete answers. Therefore, future work should include systematic testing of AI Mentor responses. A test set of programming questions, student mistakes, and AI prompts can be created to measure whether the AI gives hints, explanations, and conceptual guidance instead of full solutions. The validation and policy layers can also be improved to better detect direct answer leakage, overly specific code suggestions, and unsafe grading explanations.

Exam integrity can also be improved further. The current system includes Exam Mode, which disables AI Mentor access during exam-style assignments. This is a strong starting point, but higher-stakes exams require stronger protection. Future work can include stricter backend enforcement of exam rules, more detailed violation logging, improved fullscreen and tab-switch detection, secure exam session tracking, and clearer teacher dashboards for reviewing suspicious activity. In this way, Exam Mode

can evolve from a simple AI restriction feature into a more complete assessment integrity module.

Scalability and performance should also be tested in future work. The platform includes resource-heavy components such as browser-based code editing, Docker-based code execution, AI requests, analytics dashboards, and real-time terminal features. Since the proposal identified scalability for multiple simultaneous classroom or exam users as a challenge, future work should include load testing with concurrent students, simultaneous submissions, and multiple AI requests. Based on these results, the system can be improved with request queues, rate limiting, execution time limits, container resource limits, caching, and frontend bundle optimization.

Another future direction is automated testing and quality assurance. The implementation report lists many functional validation scenarios for student, teacher, and coordinator workflows, including login, assignment creation, code execution, AI Mentor restriction, grading, analytics, and teacher approval. In future work, these scenarios can be converted into automated unit tests, integration tests, and end-to-end tests. This would make the system easier to maintain as new features are added and would reduce the risk of breaking important workflows.

Educational evaluation is also necessary for future development. The current platform includes many learning-oriented features such as AI Mentor support, tutorials, flashcards, analytics, and feedback-based study materials. However, the real educational value of these features should be measured with students and teachers. The original proposal suggested usability tests, learning outcome comparisons, time-to-solve measurements, teacher workload reduction metrics, and user satisfaction surveys. These evaluation methods can be used in future work to understand whether the platform actually improves learning, reduces teacher workload, and supports fair assessment.

The platform can also be extended with more advanced educational features. Future versions may include personalized problem recommendations, adaptive learning paths, misconception detection, plagiarism or code similarity analysis, richer teacher analytics, peer review workflows, and integration with learning management systems such as Moodle or Blackboard. These extensions would make the system more useful in real classroom and department-level scenarios.

Finally, maintainability can be improved as the system grows. Some frontend, backend, AI service, and database components may become harder to manage as more features are added. Future work should therefore include modular refactoring, clearer API documentation, shared type definitions, better database schema documentation, and

more consistent error handling. These improvements would make the platform easier to develop, test, deploy, and extend in future semesters.

Overall, future work should focus not only on adding new features, but also on making the existing system more secure, reliable, scalable, maintainable, and educationally validated. This would help transform the current successful prototype into a stronger platform that can be used in real programming courses and assessment environments.

6. Final Remarks

The AI-Assisted Programming Platform for Education and Assessment demonstrates that AI can be integrated into programming education in a controlled and meaningful way when it is supported by proper system design, role-based workflows, academic integrity mechanisms, and teacher supervision. The project successfully combines student learning support, teacher productivity tools, automated code execution, structured grading, analytics, and coordinator-level academic management in a single full-stack prototype.

At the same time, the project also shows that building an educational AI platform requires more than implementing features. Security, fairness, scalability, AI safety, deployment reliability, and real classroom validation are essential for turning the prototype into a production-ready system. Therefore, the current implementation should be seen as a strong foundation that proves the concept and provides a clear direction for future improvement.

Overall, the project achieved its main goal of creating an AI-assisted programming platform that supports learning without replacing student problem solving. With further work on production hardening, systematic testing, AI validation, and classroom-based evaluation, the platform has the potential to become a practical and reliable tool for programming education and assessment.